

30 AI Engineer Project Problem Statements

Built to extend your existing repos — `langgraph-workflow-orchestrator` (agent orchestration) and `production-grade-rag-engine` (RAG) — into a complete portfolio. Grouped by category so you can pick a spread rather than stacking similar projects. Each entry gives you a problem statement, not a tutorial link — the point is to design and defend your own architecture decisions in an interview.

A. Agentic Systems & Multi-Step Workflows (7)

1. Support-Ticket Triage Agent

Problem: Incoming support tickets (public dataset or your own inbox) need classification, related-issue retrieval, draft response generation, and escalation of ambiguous cases to a human. **Build:** Planner/executor split, tool calling against a real ticketing API or GitHub Issues API, confidence-based escalation threshold, full audit trail of every decision. **Signal:** Founding-engineer-style ownership of a real business process end-to-end.

2. Code-Review Agent

Problem: PRs need automated first-pass review — style, obvious bugs, missing tests, security smells — before a human reviewer looks at them. **Build:** GitHub API integration via webhook, static-analysis tool calling, LLM-based semantic review, comment-posting with confidence scores, human override loop. **Signal:** Dev-tools AI is one of the hottest 2026 YC verticals — this is a direct hit.

3. Compliance Document Reviewer

Problem: Contracts or policy documents need to be checked against a rule set (e.g., GDPR clauses, internal policy) and flagged with citations. **Build:** Rule-engine + LLM hybrid, citation-grounded flagging, mandatory-human-review routing for high-risk flags, audit log. **Signal:** EU employers specifically value compliance-aware AI given the EU AI Act.

4. Multi-Agent Research Assistant

Problem: A research question needs decomposition into sub-questions, parallel web/document research per sub-question, and synthesis into a cited report. **Build:** Supervisor agent delegating to specialized research sub-agents, parallel execution (not sequential), source deduplication, citation tracking. **Signal:** Tests your ability to design non-trivial multi-agent coordination, not just single-agent tool use.

5. Autonomous Data-Pipeline Debugger

Problem: A broken ETL/data pipeline needs root-cause diagnosis — the agent inspects logs, checks recent schema changes, tests hypotheses, and proposes a fix. **Build:** Log-parsing tools, hypothesis-generation loop, sandboxed test execution, structured incident report output. **Signal:** AI-for-ops/SRE is an emerging, less crowded niche — strong differentiator.

6. Sales Outreach Qualification Agent

Problem: Inbound leads (from a public dataset or form submissions) need enrichment, qualification scoring, and a personalized first-touch draft. **Build:** Enrichment via public APIs, scoring rubric as a structured tool call, draft generation with tone/persona control, human-approval gate before send. **Signal:** Sales-agent verticals are heavily funded in the 2026 YC cohort.

7. Meeting-to-Action-Items Agent

Problem: A meeting transcript needs to become a structured set of action items assigned to people, with follow-up tracking over time. **Build:** Transcript segmentation, speaker/action extraction, structured output validation, a persistence layer that tracks follow-through across multiple meetings (state over time, not one-shot). **Signal:** Shows you can design agents with long-lived memory/state, not just single-session logic.

B. RAG & Retrieval Systems (5)

8. Multi-Source Enterprise Knowledge Assistant

Problem: Answers need to be pulled from heterogeneous sources — Slack-style chat exports, wiki pages, and code — with source-type-aware retrieval. **Build:** Separate retrieval strategies per source type, a router that decides which source(s) to query, unified citation format across source types. **Signal:** Real enterprises never have “one clean document store” — this proves you can handle that mess.

9. Long-Document / Legal Contract RAG

Problem: Long documents (100+ pages) break naive chunking — questions need answers that span multiple sections with correct cross-references. **Build:** Hierarchical chunking, document-structure-aware retrieval (section/clause-level), multi-hop retrieval for cross-referencing clauses. **Signal:** Long-context RAG is a distinct, harder skill than short-doc RAG — worth calling out explicitly on your resume.

10. Code-Aware RAG for a Real Codebase

Problem: “How does X work in this repo” needs retrieval that understands code structure (functions, imports, call graphs), not just text similarity. **Build:** AST-aware chunking, symbol-graph-informed retrieval, combining code + docstrings + commit history as separate retrieval channels. **Signal:** Dev-tools AI companies specifically test for this — a strong complement to your existing RAG engine.

11. Freshness-Aware RAG with Incremental Indexing

Problem: A knowledge base changes daily (e.g., a news source or changelog feed) — the index needs incremental updates without full re-embedding, and answers need to reflect recency. **Build:** Incremental ingestion pipeline, staleness detection, time-decay in ranking, a scheduled re-index job you can show running. **Signal:** Proves you think about RAG as an operational system, not a one-time build.

12. RAG with Active Feedback Loop

Problem: User thumbs-up/down on answers should measurably improve retrieval quality over time, not

just sit in a log. **Build:** Feedback capture, a retraining/reranking-weight-adjustment loop, before/after eval comparison showing the loop actually improved quality. **Signal:** Turns "we collect feedback" (common claim) into "here's the measured before/after," which almost nobody actually shows.

C. Evaluation, Guardrails & Safety (4)

13. Standalone LLM Eval & Red-Team Framework

Problem: Every LLM app needs a reusable way to measure faithfulness, toxicity, PII leakage, and jailbreak resistance — built once, applied to both of your existing projects. **Build:** Pluggable metric library, a self-curated golden dataset, a red-team prompt set you build and run against your own RAG/agent repos, CI integration that gates deploys on regression. **Signal:** The single highest-value project per your target roles — eval literacy is repeatedly named as the top 2026 differentiator signal.

14. Agent Behavior Auditor

Problem: Multi-step agents (like your orchestrator) can silently loop, take unintended actions, or drift off-task — you need a way to measure *that*, not just final-answer quality. **Build:** Trace-level metrics: steps-to-completion, tool-call error rate, loop detection, task-success rate against a labeled task set — applied directly to your existing LangGraph project. **Signal:** Extends your existing orchestrator repo with the evaluation layer that makes it "production-grade" rather than "a demo that works most of the time."

15. Bias & Fairness Probe for an LLM Application

Problem: An LLM-powered decision system (e.g., resume screening, loan-eligibility drafting) needs to be probed for demographic bias in its outputs. **Build:** Counterfactual prompt pairs (same query, varied demographic signals), statistical disparity measurement, a documented bias report. **Signal:** Increasingly required by EU and enterprise procurement processes — rare but valued skill.

16. Cost-of-Error Simulation for an AI Decision System

Problem: For a system that takes autonomous actions (like your agent projects), quantify the real-world cost of false positives vs. false negatives, not just accuracy. **Build:** A simulation harness that runs your existing agent against synthetic scenarios and computes expected cost under different confidence thresholds. **Signal:** Reframes "accuracy" into business-impact language — exactly how a CTO evaluates whether to trust an agent with real actions.

D. AI Infra, MLOps & Scale (6)

17. Multi-Provider LLM Router with Cost/Latency SLAs

Problem: Route each request to the cheapest model that can meet a quality bar, with automatic fallback if a provider fails. **Build:** Difficulty-classification pre-router, per-route cost/latency tracking, circuit breaker on provider failure, load-tested throughput numbers. **Signal:** Proves systems-engineering thinking under real load — the top gap tech-giant interviewers screen for.

18. Semantic Caching Layer with Measured Savings

Problem: Repeated/similar queries shouldn't hit the LLM every time — but naive caching either misses near-duplicates or serves wrong answers to subtly different questions. **Build:** Embedding-based similarity cache with a tuned threshold, cache-hit-rate dashboard, before/after cost and latency comparison under load test. **Signal:** A focused, deployable infra project you can point to with hard numbers ("¢ per 1K requests: before vs. after").

19. GPU/Inference Cost Dashboard for Self-Hosted Models

Problem: Teams running open-weight models locally (vLLM/Ollama) need visibility into GPU utilization, cost per token, and batching efficiency. **Build:** Prometheus + Grafana instrumentation on top of a self-hosted vLLM deployment, batching-strategy comparison, ¢ per million tokens computed and compared to API pricing. **Signal:** Self-hosting cost analysis is a distinct, high-value skill as companies increasingly weigh open-weight vs. API tradeoffs.

20. CI/CD Pipeline for LLM Prompts and Agents

Problem: Prompt and agent-logic changes currently ship with no automated regression check — a bad prompt edit can silently degrade quality in production. **Build:** Git-hooked eval suite (reuse Project 13), automatic rollback on regression, versioned prompt registry with diffing. **Signal:** "LLMOps" is explicitly named across 2026 job descriptions as a distinct, valued skill from generic MLOps.

21. Observability & Tracing Layer for Multi-Agent Systems

Problem: When your LangGraph orchestrator misbehaves, you currently have to read raw logs — you need structured traces showing every node, tool call, and state transition. **Build:** OpenTelemetry-style tracing wired into your existing orchestrator, a trace-visualization dashboard, alerting on anomalous traces (excess steps, repeated failures). **Signal:** Directly upgrades your existing repo into something that looks operated, not just built.

22. Classical ML Model Serving + Monitoring (Non-LLM)

Problem: A traditional ML model (e.g., churn prediction, fraud scoring) needs to be served with drift detection and automatic retraining triggers. **Build:** FastAPI serving layer, feature-drift monitoring (e.g., using a stats-based detector), scheduled retraining pipeline, before/after model-performance tracking. **Signal:** Important counterbalance — proves you're not "LLM-only," which some tech-giant ML Engineer roles specifically screen for.

E. Fine-Tuning & Model Adaptation (3)

23. Domain-Adapted Small Model via LoRA Fine-Tuning

Problem: A small open-weight model needs to match or beat a much larger general model on one narrow, well-defined task (e.g., structured extraction from a specific document type). **Build:** LoRA/QLoRA fine-tuning pipeline, a labeled dataset you built yourself, benchmark against the base model and against a

frontier API model on cost and accuracy. **Signal:** Fine-tuning is explicitly named in your target skill list and is rarer on portfolios than RAG/agents — real differentiator.

24. Distillation Pipeline: Frontier Model → Cheap Local Model

Problem: A production system currently pays frontier-model prices for a task a much smaller model could handle if trained on the right data. **Build:** Generate a training set by sampling the frontier model's outputs on your task, fine-tune a small model on it, measure quality retained vs. cost saved. **Signal:** This is exactly the "cost-optimized inference" story CTOs want to hear, told through a fine-tuning lens instead of a routing lens.

25. Structured-Output Reliability Fine-Tune

Problem: Getting a model to reliably return valid JSON/schema-conforming output under prompting alone has a non-trivial failure rate at scale. **Build:** Measure baseline schema-validation failure rate via prompting only, fine-tune (or use constrained decoding) to reduce it, report the before/after failure rate under load. **Signal:** A very "production reliability" framed project — shows you understand where prompting alone breaks down.

F. Multi-Modal AI (3)

26. Document-Image-to-Structured-Data Pipeline

Problem: Scanned invoices/forms/receipts need to become structured, validated data — not just OCR text dumped into a field. **Build:** Vision-model extraction + schema validation + confidence-based human-review routing for low-confidence extractions. **Signal:** Multi-modal document AI is a concrete, widely-deployed enterprise use case (finance, logistics, healthcare back-office).

27. Voice-to-Action Agent

Problem: A spoken instruction needs to become a validated, executed action (e.g., "schedule a follow-up with the vendor for next Tuesday") with confirmation before execution. **Build:** Speech-to-text, intent/slot extraction, a confirmation step before any real action fires, fallback to clarification when confidence is low. **Signal:** Voice-AI agents are a distinct, growing 2026 YC vertical (customer service, field service) — shows range beyond text.

28. Multi-Modal RAG (Text + Images + Tables)

Problem: Real documents mix prose, tables, and charts — a text-only RAG pipeline silently fails to answer questions that require reading a chart or table. **Build:** Separate extraction/embedding paths for text, tables, and images; a retrieval layer that can pull the right modality for a given question; citation that references the correct modality. **Signal:** Extends your existing RAG engine with a capability most portfolio RAG projects skip entirely.

G. Industrial / Vertical AI (2)

29. Predictive Safety Risk Scoring (Mining/Industrial)

Problem: Public MSHA/OSHA incident and violation data can be used to score ongoing operational risk and flag high-risk patterns before an incident occurs. **Build:** Structured-data anomaly detection + LLM-based narrative-report analysis, combined risk score, explainability layer citing which factors drove a flag, false-positive rate explicitly reported. **Signal:** Rare, memorable vertical — directly ties to your stated industrial-AI interest and stands out against a sea of generic chatbot portfolios.

30. Equipment-Failure Early-Warning System

Problem: Time-series sensor data (public industrial IoT datasets exist, e.g., NASA turbofan degradation data) needs to predict failure before it happens, with a clear lead-time-vs-false-alarm tradeoff. **Build:** Time-series anomaly/failure-prediction model, an LLM layer that turns raw predictions into a human-readable maintenance recommendation, a tunable alert threshold with cost-of-false-alarm framing. **Signal:** Classic "AI + operations" story that plays extremely well with German/EU industrial employers specifically (Siemens-style manufacturing AI market).

How to Pick From This List

You already have strong coverage in **agent orchestration** and **RAG**. To round out a 4–6 project flagship portfolio without redundancy, a strong next-three pick is:

#13 (Eval & Red-Team Framework) — apply it directly to your two existing repos. Highest ROI, lowest net-new build time.

#17 or #18 (Router or Semantic Cache) — gives you the "systems/scale" project tech giants specifically probe for.

#29 (Mining Safety Risk Scoring) — your strongest differentiator; almost nobody else will have this.

Optional 4th: **#21 (Observability layer)** — cheapest way to make your *existing* orchestrator repo look meaningfully more production-grade with a relatively small addition.

Each project above is a problem statement, not a spec — the architecture decisions you make (and can explain in an interview) are what actually generate the resume-bullet metrics from the previous strategy doc.